SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1402 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I K THARUN (192424265)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :K THARUN

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System

A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

(i) Design suitable intermediate code for the given assignment and Boolean expressions. ***(K3)***

(ii) Explain how backpatching can help in handling conditional transaction validation. ***(K4)***

(iii) Analyze the role of intermediate languages in improving banking software portability. ***(K5)***

Banking Transaction Processing System

(i) Design suitable intermediate code for the given assignment and Boolean expressions. (K3)

In a banking transaction processing system, intermediate code can be used to represent transaction details and validate transactions before execution. Three-address code is suitable because each instruction contains a simple operation and makes the code easier to optimize.

Consider the following transaction logic:

```
balance = balance + amount
if amount > 0 && balance >= amount
   process transaction
else
   reject transaction
```

A suitable intermediate representation is:

```
t1 = amount > 0
if t1 == 0 goto L2
t2 = balance >= amount
if t2 == 0 goto L2
t3 = balance + amount
balance = t3
goto L3
L2:
status = "Rejected"
```

L3:
End

The code first checks whether the transaction amount is valid. If the amount is greater than zero, the available balance is checked. If both conditions are satisfied, the balance is updated.
For transaction classification using a case statement, the intermediate code can be represented as:
if type == 1 goto Ldeposit
if type == 2 goto Lwithdraw
if type == 3 goto Lloan
goto Ldefault

Ldeposit:
call deposit(amount)
goto Lend

Lwithdraw:
call withdraw(amount)
goto Lend

Lloan:
call loan(amount)
goto Lend

Ldefault:
status = "Invalid Transaction"

Lend:
This representation allows the compiler to clearly separate different transaction operations.

(ii) Explain how backpatching can help in handling conditional transaction validation. (K4)

Backpatching is a compiler technique used to fill in the target addresses of jump instructions when the actual destinations are not known during the initial generation of intermediate code.
In banking software, conditional validation may involve several conditions such as:

if amount > 0 && balance >= amount
Initially, the compiler may generate:
100: if amount > 0 goto _
101: goto _
102: if balance >= amount goto _
103: goto _

The blank targets are filled later when the compiler knows where the valid and invalid transaction blocks are located.

For example:
100: if amount > 0 goto 102
101: goto Lreject
102: if balance >= amount goto Lprocess
103: goto Lreject

Backpatching ensures that all conditional branches point to the correct transaction-processing or rejection block.

This is important in banking systems because incorrect branching could cause a valid transaction to be rejected or an invalid transaction to be processed. Therefore, accurate backpatching improves the correctness and reliability of transaction validation.

(iv) **Analyze the role of intermediate languages in improving banking software portability. (K5)**

An intermediate language acts as a layer between the source program and the target machine code. Instead of directly translating the banking scripting language into one particular processor's machine code, the compiler first generates intermediate code.

The same intermediate representation can then be translated into machine code for different platforms.
For example:
Source Banking Language
↓
Intermediate Code
↓
Machine Code

This improves portability because the banking application does not need to be completely rewritten for every processor or operating system.
Intermediate languages also provide a common representation for optimization. The compiler can perform common subexpression elimination, constant folding, dead-code elimination, and other optimizations before generating platform-specific code.
Therefore, intermediate languages improve **portability, maintainability, optimization, and scalability** of banking software. They are particularly useful when banking applications need to operate across servers, cloud platforms, ATMs, and other computing environments.

2. Smart Traffic Control System

An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.

Tasks:

(i) Construct intermediate code for the traffic signal control logic. ***(K3)***

(ii) Explain how procedure calls improve modularity in the traffic control application. ***(K4)***

(iii) Evaluate the importance of compiler optimization in real-time embedded systems. ***(K5)***

Smart Traffic Control System

(i) Construct intermediate code for the traffic signal control logic. (K3)

Consider a traffic control system where the signal depends on traffic density.

Assume:

- density > 80 → Red signal
- density > 50 → Yellow signal
- Otherwise → Green signal

The intermediate code can be:

```
if density > 80 goto Lred
if density > 50 goto Lyellow
goto Lgreen

Lred:
signal = RED
goto Lend

Lyellow:
signal = YELLOW
goto Lend

Lgreen:
signal = GREEN

Lend:
```

The compiler first checks whether the traffic density is greater than 80. If so, the red signal is selected. Otherwise, it checks whether the density is greater than 50. If that condition is also false, the green signal is selected.

For more complex conditions, Boolean expressions can also be represented using temporary variables and conditional jumps.

(ii) Explain how procedure calls improve modularity in the traffic control application. (K4)

Procedure calls allow the traffic-control application to divide a large program into smaller and independent modules.

For example:

call readTrafficDensity()
call analyzeTraffic()
call controlSignal()
call updateDisplay()

Each procedure performs a specific task.

readTrafficDensity() obtains traffic information, analyzeTraffic() determines the traffic condition, and controlSignal() selects the appropriate traffic signal.

This provides several benefits:

1. **Modularity:** Each function handles one specific operation.
2. **Code reuse:** The same procedure can be used for multiple traffic signals.
3. **Easy maintenance:** Changes can be made to one procedure without modifying the entire program.
4. **Reduced complexity:** Large programs are divided into smaller units.
5. **Better testing:** Each procedure can be tested independently.
6. **Improved readability:** The program becomes easier to understand.

Thus, procedure calls help organize the traffic control software into reusable and manageable components.

(iv) **Evaluate the importance of compiler optimization in real-time embedded systems. (K5)**

Compiler optimization is extremely important in real-time embedded systems because these systems must respond within strict time limits.

In a traffic control system, inefficient intermediate code may cause delays in detecting traffic conditions and changing signals. Optimization reduces unnecessary instructions and improves execution speed.

Important optimizations include:

- Common subexpression elimination.
- Constant folding.
- Dead-code elimination.
- Efficient register allocation.
- Reduction of unnecessary procedure calls.
- Optimization of conditional branches.

Therefore, compiler optimization helps achieve **faster response time, lower resource usage, better performance, and more predictable execution**, making it essential for real-time traffic control systems.

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.

Tasks:

(i) Generate intermediate representation for the recommendation logic. ***(K3)***

(ii) Discuss how case statements can efficiently handle product category selection. ***(K4)***

(iii) Analyze the role of compiler design in improving large-scale e-commerce applications. ***(K5)***

(i) Generate intermediate representation for the recommendation logic. (K3)

Consider a recommendation system that selects products according to customer category and purchase amount.

For example:

```
if category == 1 && amount > 5000
recommend electronics
else if category == 2
   recommend clothing
else
   recommend general products
```

A suitable intermediate representation is:

```
if category == 1 goto L1
if category == 2 goto Lclothing
goto Lgeneral

L1:
if amount > 5000 goto Lelectronics
goto Lgeneral

Lelectronics:
call recommendElectronics()
goto Lend

Lclothing:
call recommendClothing()
goto Lend
```

Lgeneral:
call recommendGeneral()

Lend:
This intermediate representation converts high-level recommendation rules into simple conditional jumps and procedure calls.

(ii) Discuss how case statements can efficiently handle product category selection. (K4)

Case statements are useful when a recommendation engine has many product categories.
For example:

```
switch(category)

case 1:
   recommend electronics

case 2:
   recommend clothing

case 3:
   recommend books

case 4:
   recommend sports

default:
   recommend general products
```

The compiler can convert the case statement into conditional jumps or a jump table.
A simplified intermediate representation is:

```
if category == 1 goto Lelectronics
if category == 2 goto Lclothing
if category == 3 goto Lbooks
if category == 4 goto Lsports
goto Lgeneral
```

For a large number of consecutive case values, a jump table can provide faster selection because the required destination can be accessed directly using the case value.

Case statements therefore improve the organization of product-selection logic and can provide efficient control flow when many categories are present.

(iii) Analyze the role of compiler design in improving large-scale e-commerce applications. (K5)

Compiler design plays an important role in improving the performance and maintainability of large-scale e-commerce applications.

An e-commerce application may process millions of product searches, customer requests, recommendations, and transactions. Efficient intermediate code generation and optimization can reduce the time required to process these operations.

Compiler optimization techniques such as common subexpression elimination, constant folding, dead-code elimination, and efficient control-flow generation can reduce unnecessary computation.

Intermediate representation also allows the compiler to optimize the recommendation rules independently of the target hardware.

A well-designed compiler provides:

1. Faster execution of recommendation algorithms.
2. Efficient memory usage.
3. Better portability across different platforms.
4. Easier maintenance of business rules.
5. Improved scalability.
6. Reduced computational overhead.
7. Better utilization of server resources.

Therefore, compiler design contributes significantly to the **speed, scalability, portability, and maintainability** of large-scale e-commerce applications.

4.Hospital Patient Monitoring System

A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.

Tasks:

(i) **Design intermediate code for patient condition monitoring logic.** ***(K3)***
(ii) **Explain the significance of backpatching in emergency alert systems.** ***(K4)***
(iii) **Evaluate how compiler optimization improves reliability in healthcare applications.** ***(K5)***

(i) Design intermediate code for patient condition monitoring logic. (K3)
Consider a patient monitoring system that checks heart rate and oxygen level.
Assume an emergency alert is generated when:
heartRate > 120 || oxygenLevel < 90
The intermediate code can be:
if heartRate > 120 goto Lemergency
if oxygenLevel < 90 goto Lemergency
goto Lnormal

Lemergency:
alert = 1
call emergencyAlert()
goto Lend

Lnormal:
alert = 0

Lend:
The compiler first checks the heart rate. If it indicates an emergency condition, the system immediately jumps to the emergency block. Otherwise, the oxygen level is checked.
This representation supports quick decision-making and avoids unnecessary condition checking.

(ii) Explain the significance of backpatching in emergency alert systems. (K4)
Backpatching is important when the compiler generates conditional branches whose final target addresses are not immediately known.
For example:
100: if heartRate > 120 goto _
101: if oxygenLevel < 90 goto _
102: goto _

The compiler maintains lists containing the incomplete jump instructions.
The true conditions can be connected to the emergency alert block,
while the false condition can be connected to the normal monitoring block.

After the target locations are determined, backpatching fills
in the addresses:
100: if heartRate > 120 goto Lemergency
101: if oxygenLevel < 90 goto Lemergency
102: goto Lnormal

Correct backpatching ensures that emergency conditions reach the alert routine without being incorrectly directed to the normal-processing section. In a patient monitoring system, accurate control flow is particularly important because delays or incorrect branching could affect the timely generation of alerts.

(iii) Evaluate how compiler optimization improves reliability in healthcare applications. (K5)

Compiler optimization can improve both performance and reliability in healthcare applications by reducing unnecessary computations and making execution more predictable.

For example, if a vital-sign condition is repeatedly calculated, the compiler can reuse an existing result instead of recalculating it.

Optimization can:

1. Reduce execution time.
2. Remove unnecessary instructions.
3. Reduce processor workload.
4. Improve memory utilization.
5. Make real-time responses more predictable.
6. Reduce unnecessary control-flow operations.
7. Improve the efficiency of embedded monitoring devices.

However, optimization must preserve the original program's meaning. In healthcare systems, correctness is more important than simply making the program faster.

Therefore, compiler optimization should be carefully validated and tested so that improved performance does not introduce incorrect medical decisions or alert behavior.

5.Online Airline Reservation System

An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.

Tasks:

(iv) Develop intermediate code for ticket booking and seat allocation operations. *(K3)*

(v) Explain how procedure calls support modular reservation management. *(K4)*

(vi) Analyze the role of intermediate languages in distributed airline reservation systems. *(K5)*

(i) Develop intermediate code for ticket booking and seat allocation operations. (K3)

Consider an airline reservation system where a customer selects a seat and books a ticket.
A suitable intermediate code is:

```
if seatAvailable == 1 goto Lavailable
goto Lunavailable

Lavailable:
t1 = seatNumber
t2 = ticketPrice
call reserveSeat(t1)
call processPayment(t2)
status = "Booking Confirmed"
goto Lend

Lunavailable:
status = "Seat Unavailable"

Lend:
```

The code first checks whether the requested seat is available. If it is available, the seat is reserved and payment processing is performed. After successful processing, the booking status is updated.
A more detailed representation can be:

```
t1 = selectedSeat
t2 = passengerID
if seatAvailable == 0 goto Lerror
call allocateSeat(t1, t2)
call confirmPayment()
status = "Confirmed"
goto Lend

Lerror:
status = "Booking Failed"

Lend:
```

This intermediate representation separates individual operations and makes them easier for the compiler to optimize.

(ii) Explain how procedure calls support modular reservation management. (K4)

Procedure calls allow the airline reservation system to divide its functionality into independent modules.
For example:
call searchFlight()
call checkSeatAvailability()
call allocateSeat()
call processPayment()
call generateTicket()
call sendConfirmation()
Each procedure performs a specific task.
For example, checkSeatAvailability() verifies available seats, allocateSeat() assigns the selected seat, and processPayment() handles payment processing.

Procedure calls provide the following advantages:

1. **Modularity:** Each reservation operation is implemented separately.
2. **Code reuse:** Procedures can be called for many different flights.
3. **Easy maintenance:** Individual modules can be modified independently.
4. **Better testing:** Each procedure can be tested separately.
5. **Reduced complexity:** The overall reservation system becomes easier to manage.
6. **Improved organization:** Different reservation functions remain logically separated.

Thus, procedure calls make the reservation system more modular, maintainable, and scalable.

(iii) Analyze the role of intermediate languages in distributed airline reservation systems. (K5)
An intermediate language provides a common representation between the airline's high-level reservation language and the target machine or platform.
The compilation process can be represented as:
Reservation Program
↓
Intermediate Representation
↓
Platform-Specific Code
↓
Execution
This is especially useful in distributed airline reservation systems because different components may run on different hardware, operating systems, or cloud platforms.

The intermediate language provides a common form that can be optimized before generating platform-specific code.

It also improves portability because the same reservation application can be adapted to different systems without completely rewriting the source program. Intermediate languages also support compiler optimizations that can improve the processing of:

- Flight searches.
- Seat availability checks.
- Seat allocation.
- Payment confirmation.
- Ticket generation.
- Passenger information processing.

In a large-scale distributed system, efficient intermediate code can reduce processing time and resource consumption. This is important when thousands of users are booking tickets simultaneously.

Conclusion

Intermediate languages provide **portability, optimization, scalability, and maintainability** for distributed airline reservation systems. They allow the same high-level reservation logic to be efficiently adapted to different platforms while helping the compiler optimize execution for large numbers of concurrent booking requests.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation